Every Core Algorithm in Code

Career preparation notes Sources: standard material, listed under References

What this file is really about:

$$\text{the maths from the lectures} \rightarrow \text{a from-scratch implementation} \rightarrow \text{a check against scikit-learn}$$

How to use this file. Each block in Part B runs by itself. Read the corresponding lecture, close this page, write the algorithm yourself, then compare. Every block ends by checking its answer against a known result or against scikit-learn.

Step-up path

You need. The maths of the matching lecture.

1. Write linear models from their equations.
2. Build a tree, a forest and boosting.
3. Implement neighbours and naive Bayes.
4. Write clustering and PCA.
5. Compare everything through one interface.

You can stop when you can implement the core algorithms from memory and check them against a library.

Contents

Part A Choosing and Costing

A1 Which algorithm first

Situation	Start with	Why
Tabular, a few thousand rows or more	Gradient-boosted trees	Strong with little feature work
Need to explain each coefficient	Linear or logistic regression	Directly interpretable
Very many sparse features	Regularised linear model	Fast and hard to overfit
Small data, non-linear boundary	Kernel SVM or a random forest	Works without a huge sample
Images, audio, text	Neural networks, usually pretrained	Learn the representation
No labels, want groups	K-means, then a Gaussian mixture	Simple baseline first
Rare events	Isolation forest or a density model	Models normal behaviour

A2 What each one costs

With n rows and p features:

Algorithm	Training	Prediction
Linear regression, normal equation	$O(np^2 + p^3)$	$O(p)$
Logistic regression, one Newton step	$O(np^2 + p^3)$	$O(p)$
Decision tree	$O(pn \log n)$ for balanced trees	$O(\text{depth})$
Random forest, T trees	T times one tree	T times one tree
k-nearest neighbours	none	$O(np)$ by brute force
Kernel SVM	between $O(n^2)$ and $O(n^3)$	$O(\text{support vectors})$
K-means, t iterations	$O(nkpt)$	$O(kp)$
PCA, full	$O(np^2 + p^3)$	$O(pk)$

A3 Which need scaled features

Distance-based and gradient-based methods need scaling: KNN, K-means, SVM, PCA, neural networks and regularised linear models. Trees do not, since only the order of each feature matters.

Part B In Code

B1 Linear regression

```
import numpy as np
from sklearn.linear_model import LinearRegression

rng = np.random.default_rng(0)
X = rng.normal(size=(200, 3)); y = X @ np.array([1.5, -2.0, 0.5]) + 4 + 0.1 * rng.normal(si
A = np.c_[np.ones(200), X]

normal = np.linalg.solve(A.T @ A, A.T @ y)

w = np.zeros(4)
for _ in range(2000):
    w -= 0.1 * 2 * A.T @ (A @ w - y) / len(y)

sk = LinearRegression().fit(X, y)
assert np.allclose(normal, np.r_[sk.intercept_, sk.coef_]) and np.allclose(w, normal, atol=
print(normal.round(3))
```

B2 Ridge regression

The intercept is not penalised, which is the same as centring the data first.

```
import numpy as np
from sklearn.linear_model import Ridge

def ridge(X, y, lam):
    Xc, yc = X - X.mean(0), y - y.mean()
    w = np.linalg.solve(Xc.T @ Xc + lam * np.eye(X.shape[1]), Xc.T @ yc)
    return y.mean() - X.mean(0) @ w, w

rng = np.random.default_rng(0)
X = rng.normal(size=(100, 5)); y = X[:, 0] - 2 * X[:, 1] + rng.normal(size=100) + 3
b, w = ridge(X, y, 10.0)
sk = Ridge(alpha=10.0).fit(X, y)
assert abs(b - sk.intercept_) < 1e-8 and np.allclose(w, sk.coef_)
print(round(b, 3), w.round(3))
```

B3 Lasso by coordinate descent

```
import numpy as np
from sklearn.linear_model import Lasso

def lasso(X, y, lam, sweeps=2000):
    n, p = X.shape
    w = np.zeros(p); r = y.copy()
    z = (X ** 2).sum(0) / n
    for _ in range(sweeps):
        for j in range(p):
            r += X[:, j] * w[j]                          # remove feature j's fit
            rho = X[:, j] @ r / n
            w[j] = np.sign(rho) * max(abs(rho) - lam, 0) / z[j]   # soft threshold
            r -= X[:, j] * w[j]
    return w

rng = np.random.default_rng(0)
X = rng.normal(size=(200, 10)); y = 3 * X[:, 0] - 2 * X[:, 3] + 0.5 * rng.normal(size=200)
X, y = X - X.mean(0), y - y.mean()
w = lasso(X, y, 0.2)
sk = Lasso(alpha=0.2, fit_intercept=False, tol=1e-10, max_iter=100000).fit(X, y)
assert np.allclose(w, sk.coef_, atol=1e-5)
print('non-zero features', np.flatnonzero(w))
```

B4 Logistic regression by Newton's method

```
import numpy as np
from sklearn.linear_model import LogisticRegression

def newton_logistic(X, y, lam=1e-4, steps=25):
    A = np.c_[np.ones(len(X)), X]
    w = np.zeros(A.shape[1])
    reg = lam * np.eye(len(w)); reg[0, 0] = 0
    for _ in range(steps):
        p = 1 / (1 + np.exp(-A @ w))
        g = A.T @ (p - y) + reg @ w
        H = (A.T * (p * (1 - p))) @ A + reg
        w -= np.linalg.solve(H, g)
    return w

rng = np.random.default_rng(0)
X = rng.normal(size=(500, 3))
y = (rng.random(500) < 1 / (1 + np.exp(-(X[:, 0] - 2 * X[:, 1] + 0.5)))).astype(float)
w = newton_logistic(X, y)
sk = LogisticRegression(C=1e4, tol=1e-10, max_iter=5000).fit(X, y)
assert np.allclose(w, np.r_[sk.intercept_, sk.coef_[0]], atol=1e-3)
print(w.round(3))
```

B5 Softmax regression

```
import numpy as np
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)
X = (X - X.mean(0)) / X.std(0)
A = np.c_[np.ones(len(X)), X]
W = np.zeros((A.shape[1], 3))
one_hot = np.eye(3)[y]

for _ in range(2000):
    z = A @ W
    p = np.exp(z - z.max(1, keepdims=True))
    p /= p.sum(1, keepdims=True)
    W -= 0.1 * A.T @ (p - one_hot) / len(y)

acc = ((A @ W).argmax(1) == y).mean()
print('accuracy', round(acc, 3))
assert acc > 0.95
```

B6 A linear support vector machine

Sub-gradient descent on the regularised hinge loss.

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.svm import SVC

def pegasos(X, y, lam=0.01, epochs=100, seed=0):
    rng = np.random.default_rng(seed)
    w, b, t = np.zeros(X.shape[1]), 0.0, 0
    for _ in range(epochs):
        for i in rng.permutation(len(X)):
            t += 1
            eta = 1 / (lam * t)
            if y[i] * (X[i] @ w + b) < 1:
                w = (1 - eta * lam) * w + eta * y[i] * X[i]
                b += eta * y[i] * 0.1                          # a slower rate keeps the bia
            else:
                w = (1 - eta * lam) * w
    return w, b

X, y = make_blobs(300, centers=2, cluster_std=1.2, random_state=1)
X = (X - X.mean(0)) / X.std(0); y = 2 * y - 1
w, b = pegasos(X, y)
ours = (np.sign(X @ w + b) == y).mean()
theirs = SVC(kernel='linear', C=1).fit(X, y).score(X, y)
print(round(ours, 3), round(theirs, 3))
assert ours > theirs - 0.02
```

B7 A decision tree

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier


class Tree:
    def __init__(self, depth=4, min_leaf=2, features=None, seed=0):
        self.depth, self.min_leaf, self.features = depth, min_leaf, features
        self.rng = np.random.default_rng(seed)

    @staticmethod
    def best(x, y, min_leaf):
        order = np.argsort(x); x, y = x[order], y[order]
        n, total, ones = len(y), y.sum(), np.cumsum(y)
        lefts = np.arange(1, n)
        l1, r1 = ones[:-1], total - ones[:-1]
        rights = n - lefts
        gini = lefts * (2 * (l1 / lefts) * (1 - l1 / lefts)) + rights * (2 * (r1 / rights)
        gini = gini.astype(float)
        gini[(x[1:] == x[:-1]) | (lefts < min_leaf) | (rights < min_leaf)] = np.inf
        i = int(gini.argmin())
        return gini[i] / n, (x[i] + x[i + 1]) / 2

    def grow(self, X, y, d):
        node = dict(value=float(y.mean()))
        if d == self.depth or y.min() == y.max() or len(y) < 2 * self.min_leaf:
            return node
        p = X.shape[1]
        cols = self.rng.choice(p, self.features, replace=False) if self.features else range
        score, feat, thr = np.inf, None, None
        for j in cols:
            s, t = self.best(X[:, j], y, self.min_leaf)
```

```
            if s < score:
                score, feat, thr = s, j, t
        if feat is None or not np.isfinite(score):
            return node
        mask = X[:, feat] <= thr
        node.update(feat=feat, thr=thr, left=self.grow(X[mask], y[mask], d + 1), right=self
        return node

    def fit(self, X, y):
        self.root = self.grow(X, y, 0)
        return self

    def predict_proba(self, X):
        out = []
        for row in X:
            n = self.root
            while 'feat' in n:
                n = n['left'] if row[n['feat']] <= n['thr'] else n['right']
            out.append(n['value'])
        return np.array(out)

    def predict(self, X):
        return (self.predict_proba(X) >= 0.5).astype(int)


X, y = load_breast_cancer(return_X_y=True)
Xa, Xb, ya, yb = train_test_split(X, y, random_state=0)
ours = (Tree(depth=4).fit(Xa, ya).predict(Xb) == yb).mean()
theirs = DecisionTreeClassifier(max_depth=4, random_state=0).fit(Xa, ya).score(Xb, yb)
print(round(ours, 3), round(theirs, 3))
assert abs(ours - theirs) < 0.05
```

B8 A random forest

This block reuses the `Tree` class from B7, so run that first.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
Xa, Xb, ya, yb = train_test_split(X, y, random_state=0)
rng = np.random.default_rng(0)
trees, oob_votes, oob_counts = [], np.zeros(len(Xa)), np.zeros(len(Xa))

for t in range(40):
    i = rng.integers(len(Xa), size=len(Xa))
    tree = Tree(depth=8, features=int(np.sqrt(X.shape[1])), seed=t).fit(Xa[i], ya[i])
    trees.append(tree)
    out = np.setdiff1d(np.arange(len(Xa)), i)         # rows this tree never saw
    oob_votes[out] += tree.predict_proba(Xa[out]); oob_counts[out] += 1

seen = oob_counts > 0
oob = ((oob_votes[seen] / oob_counts[seen] >= 0.5) == ya[seen]).mean()
ours = ((np.mean([t.predict_proba(Xb) for t in trees], 0) >= 0.5) == yb).mean()
theirs = RandomForestClassifier(40, random_state=0).fit(Xa, ya).score(Xb, yb)
print('out-of-bag', round(oob, 3), 'test', round(ours, 3), 'library', round(theirs, 3))
assert ours > theirs - 0.05
```

B9 AdaBoost

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True); y = 2 * y - 1
Xa, Xb, ya, yb = train_test_split(X, y, random_state=0)
w = np.ones(len(Xa)) / len(Xa)
stumps, alphas = [], []

for _ in range(60):
    s = DecisionTreeClassifier(max_depth=1).fit(Xa, ya, sample_weight=w)
    pred = s.predict(Xa)
    err = max(w[pred != ya].sum(), 1e-12)
    if err >= 0.5:
        break
    a = 0.5 * np.log((1 - err) / err)
    w = w * np.exp(-a * ya * pred)
    w /= w.sum()
    stumps.append(s); alphas.append(a)

score = sum(a * s.predict(Xb) for a, s in zip(alphas, stumps))
acc = (np.sign(score) == yb).mean()
print('accuracy', round(acc, 3))
assert acc > 0.93
```

B10 Gradient boosting

```
import numpy as np
from sklearn.datasets import make_friedman1
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

X, y = make_friedman1(1500, noise=1.0, random_state=0)
Xa, Xb, ya, yb = train_test_split(X, y, random_state=0)
lr, base = 0.1, ya.mean()
pa, pb = np.full(len(ya), base), np.full(len(yb), base)

for _ in range(200):
    tree = DecisionTreeRegressor(max_depth=3, random_state=0).fit(Xa, ya - pa)     # residu
    pa += lr * tree.predict(Xa)
    pb += lr * tree.predict(Xb)

ours = ((pb - yb) ** 2).mean()
sk = GradientBoostingRegressor(n_estimators=200, learning_rate=0.1, max_depth=3, random_sta
theirs = ((sk.predict(Xb) - yb) ** 2).mean()
print(round(ours, 3), round(theirs, 3))
assert abs(ours - theirs) < 0.3
```

B11 Nearest neighbours

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)
Xa, Xb, ya, yb = train_test_split(X, y, random_state=0)

def knn(Xa, ya, Xb, k=5):
    d = ((Xb[:, None] - Xa[None]) ** 2).sum(-1)
    nearest = ya[d.argsort(1)[:, :k]]
    return np.array([np.bincount(r).argmax() for r in nearest])

ours = (knn(Xa, ya, Xb) == yb).mean()
theirs = KNeighborsClassifier(5).fit(Xa, ya).score(Xb, yb)
print(round(ours, 3), round(theirs, 3))
assert abs(ours - theirs) < 0.03
```

B12 Gaussian naive Bayes

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB

X, y = load_iris(return_X_y=True)
classes = np.unique(y)
prior = np.array([(y == c).mean() for c in classes])
mu = np.array([X[y == c].mean(0) for c in classes])
var = np.array([X[y == c].var(0) for c in classes]) + 1e-9 * X.var(0).max()
log_post = np.log(prior) - 0.5 * (np.log(2 * np.pi * var) + (X[:, None] - mu) ** 2 / var).s
assert (log_post.argmax(1) == GaussianNB().fit(X, y).predict(X)).all()
print('accuracy', round((log_post.argmax(1) == y).mean(), 3))
```

B13 K-means with careful starts

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

def plusplus(X, k, rng):
    C = [X[rng.integers(len(X))]]
    for _ in range(k - 1):
        d = ((X[:, None] - np.array(C)[None]) ** 2).sum(-1).min(1)
        C.append(X[rng.choice(len(X), p=d / d.sum())])
    return np.array(C)

def kmeans(X, k, seed=0, iters=100):
    rng = np.random.default_rng(seed)
    C = plusplus(X, k, rng)
    for _ in range(iters):
        L = ((X[:, None] - C[None]) ** 2).sum(-1).argmin(1)
        new = np.array([X[L == j].mean(0) if (L == j).any() else C[j] for j in range(k)])
        if np.allclose(new, C):
            break
        C = new
    return C, L, ((X - C[L]) ** 2).sum()

X, _ = make_blobs(600, centers=4, random_state=3)
best = min((kmeans(X, 4, s) for s in range(10)), key=lambda r: r[2])
sk = KMeans(4, n_init=10, random_state=0).fit(X)
print(round(best[2], 2), round(sk.inertia_, 2))
assert abs(best[2] - sk.inertia_) / sk.inertia_ < 0.01
```

B14 A Gaussian mixture by EM

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture


def em(X, k, iters=200, seed=0):
    rng = np.random.default_rng(seed)
    n, p = X.shape
    mu = X[rng.choice(n, k, replace=False)]
    S = np.array([np.cov(X.T) for _ in range(k)])
    pi = np.ones(k) / k
    for _ in range(iters):
        log_r = np.zeros((n, k))
        for j in range(k):                                  # the E step
            d = X - mu[j]
            log_r[:, j] = np.log(pi[j]) - 0.5 * ((d @ np.linalg.inv(S[j])) * d).sum(1) - 0.
        log_r -= log_r.max(1, keepdims=True)
        r = np.exp(log_r); r /= r.sum(1, keepdims=True)
        N = r.sum(0)                                        # the M step
        pi, mu = N / n, r.T @ X / N[:, None]
        for j in range(k):
            d = X - mu[j]
            S[j] = (r[:, j, None] * d).T @ d / N[j] + 1e-6 * np.eye(p)
    return pi, mu, S


X, _ = make_blobs(800, centers=3, random_state=2)
pi, mu, S = em(X, 3)
sk = GaussianMixture(3, random_state=0).fit(X)
distance = np.abs(np.sort(mu[:, 0]) - np.sort(sk.means_[:, 0])).max()
print('largest mean difference', round(distance, 4))
assert distance < 0.05
```

B15 Hierarchical clustering

```
import numpy as np
from sklearn.cluster import AgglomerativeClustering
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score

def single_link(X, k):
    D = np.sqrt(((X[:, None] - X[None]) ** 2).sum(-1))
    np.fill_diagonal(D, np.inf)
    label = np.arange(len(X)); n = len(X)
    for a, b in zip(*np.unravel_index(np.argsort(D, axis=None), D.shape)):
        if n == k:
            break
        if a < b and label[a] != label[b]:
            label[label == label[b]] = label[a]
            n -= 1
    return np.unique(label, return_inverse=True)[1]

X, _ = make_blobs(200, centers=3, cluster_std=0.5, random_state=0)
ari = adjusted_rand_score(single_link(X, 3), AgglomerativeClustering(3, linkage='single').f
print('agreement with the library', round(ari, 3))
assert ari == 1
```

B16 DBSCAN

```
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons
from sklearn.metrics import adjusted_rand_score

def dbscan(X, eps, min_pts):
    near = [np.flatnonzero(r) for r in ((X[:, None] - X[None]) ** 2).sum(-1) <= eps ** 2]
    core = np.array([len(n) >= min_pts for n in near])
    label = -np.ones(len(X), int); c = -1
    for i in range(len(X)):
        if label[i] != -1 or not core[i]:
            continue
        c += 1; label[i] = c; stack = [i]
        while stack:
            j = stack.pop()
            if not core[j]:
                continue
            for n in near[j]:
                if label[n] == -1:
                    label[n] = c; stack.append(n)
    return label

X, _ = make_moons(400, noise=0.05, random_state=0)
ari = adjusted_rand_score(dbscan(X, 0.15, 5), DBSCAN(0.15, min_samples=5).fit_predict(X))
print('agreement with the library', round(ari, 3))
assert ari > 0.99
```

B17 PCA

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

X = load_iris().data
Xc = X - X.mean(0)
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
explained = s ** 2 / (s ** 2).sum()
Z = Xc @ Vt[:2].T
sk = PCA(2).fit(X)
assert np.allclose(explained[:2], sk.explained_variance_ratio_)
assert np.allclose(np.abs(Z), np.abs(sk.transform(X)))
print('variance explained', explained[:2].round(3))
```

B18 Cross-validation, written out

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
rng = np.random.default_rng(0)
folds = np.array_split(rng.permutation(len(X)), 5)
scores = []

for k in range(5):
    test = folds[k]
    train = np.concatenate([folds[i] for i in range(5) if i != k])
    scaler = StandardScaler().fit(X[train])              # fitted on training rows only
    model = LogisticRegression(max_iter=2000).fit(scaler.transform(X[train]), y[train])
    scores.append(model.score(scaler.transform(X[test]), y[test]))

print(round(np.mean(scores), 3), '+/-', round(np.std(scores), 3))
assert np.mean(scores) > 0.95
```

B19 Anomaly detection

```
import numpy as np
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(0)
normal = rng.multivariate_normal([0, 0], [[1, 0.8], [0.8, 1]], 500)
outliers = np.array([[3.0, -3.0], [-3.0, 3.0], [5.0, -4.0]])
X = np.r_[normal, outliers]

m, S = normal.mean(0), np.cov(normal.T)
d = ((X - m) @ np.linalg.inv(S) * (X - m)).sum(1)          # squared Mahalanobis distance
assert set(np.argsort(-d)[:3]) == {500, 501, 502}

score = -IsolationForest(random_state=0).fit(X).score_samples(X)
print('largest distance rows', np.argsort(-d)[:3], '| isolation forest rank of the outliers
```

B20 All of them through one interface

Every scikit-learn model shares `fit`, `predict` and `score`, so comparing them takes very little code.

```
import warnings

from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

warnings.filterwarnings('ignore')
X, y = load_breast_cancer(return_X_y=True)
models = {
    'logistic': make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
    'svm': make_pipeline(StandardScaler(), SVC()),
    'knn': make_pipeline(StandardScaler(), KNeighborsClassifier()),
    'naive bayes': GaussianNB(),
    'tree': DecisionTreeClassifier(max_depth=4, random_state=0),
    'forest': RandomForestClassifier(100, random_state=0),
    'boosting': GradientBoostingClassifier(random_state=0),
}
results = {name: round(cross_val_score(m, X, y, cv=5).mean(), 3) for name, m in models.item
print(results)
assert min(results.values()) > 0.9
```

Part C Practice

P1. Why is the intercept usually left out of the ridge penalty?

Penalising it would make predictions depend on the origin of y, so shifting all targets would change the fit.

P2. What does the soft-threshold step do when $|\rho| \le \lambda$?

It sets that weight to exactly zero, which is how the lasso selects features.

P3. Why does the random forest above consider only some features at each split?

It makes trees less correlated, and averaging decorrelated trees reduces variance further.

P4. AdaBoost has a stump with weighted error 0.1. Find its α.

$\frac{1}{2}\ln 9 = 1.099$.

P5. In cross-validation, why is the scaler fitted inside the loop?

Fitting on all rows would leak information from the test fold.